



arquitetura_projeto

Batalha Naval — Arquitetura e Planejamento do Projeto

Disciplina: Projeto acadêmico (equipe de 4 integrantes, ~41 semanas)

Stack: Ruby, Gosu (interface gráfica), SQLite3, POO, MVC simples

1. Visão Geral

O objetivo é entregar um jogo **funcional, organizado e bem documentado** — não uma arquitetura corporativa. Por isso:

- MVC simples, sem camadas extras (sem Repository, sem ORM, sem DDD/Clean Architecture).
- Apenas os padrões de projeto que agregam valor real: **Composição** (Board é composto por Cells e Ships), **Polimorfismo** (diferentes tipos de arma), e **Strategy** (cálculo de área de efeito das armas especiais).
- Banco de dados mínimo necessário, sem ORM (acesso via SQL puro com o gem `sqlite3`).

Nota de compatibilidade: o PDF oficial ("Etapa 2 — Especificações do Jogo") pede tabuleiro mínimo 5×5, pelo menos 3 navios pequenos/2 médios/1 grande, entrada por linha/columna, e derrota por esgotamento de tentativas. O escopo que você trouxe é mais rico (3 mapas, mouse, armas especiais) e **atende ou supera** o mínimo oficial, exceto em um ponto que fica em aberto — veja a seção 12.

2. Estrutura de Pastas

```
batalha_naval/  
├─ assets/           # imagens, sons e fontes usados pelo Gosu  
├─ database/        # arquivo .sqlite3 + script schema.sql  
├─ models/          # classes de domínio (Game, Player, Board, Ship, Cell, Weapon)  
├─ views/           # telas (MainWindow, MenuView, GameView, RankingView...)  
├─ controllers/     # orquestram Model <-> View (GameController, MenuController)  
├─ utils/           # utilitários transversais (Database, ScoreCalculator)  
├─ main.rb          # ponto de entrada, inicializa MainWindow  
├─ Gemfile           # dependências (gosu, sqlite3)  
└─ README.md        # documentação do projeto
```

Pasta	Função
<code>assets/</code>	Recursos estáticos (imagens de navios, sons de explosão, fonte usada nos textos). Não contém lógica.
<code>database/</code>	Guarda o arquivo <code>.sqlite3</code> gerado e o script <code>schema.sql</code> com os <code>CREATE TABLE</code> .
<code>models/</code>	Representa o domínio do jogo — estado e regras, sem saber desenhar nada na tela.
<code>views/</code>	Toda a renderização Gosu (desenho de tabuleiro, botões, texto) e captura de eventos de mouse.
<code>controllers/</code>	Recebe eventos da View, chama os Models, decide o que a View deve mostrar em seguida.
<code>utils/</code>	Código de suporte usado por mais de uma camada (acesso a banco, cálculo de pontuação).
<code>main.rb</code>	Cria a janela principal e inicia o loop do Gosu.

3. Fluxo MVC

```

Mouse click (View)
  → View repassa coordenadas ao Controller
    → Controller chama método do Model (ex: board.attack(x, y))
      → Model atualiza seu estado interno e retorna resultado
    → Controller decide próxima tela/estado
  → View re-renderiza com o novo estado do Model

```

A View **nunca** altera o Model diretamente; o Controller é o único intermediário. Isso é suficiente para o escopo — não é necessário Observer/Event Bus.

4. Classes

4.1 Models

Classe	Atributos principais	Responsabilidade
<code>Game</code>	<code>board_player</code> , <code>board_enemy</code> , <code>current_turn</code> , <code>state</code>	Orquestra a partida: turno atual, verifica vitória/derrota, aplica pontuação ao final.
<code>Player</code>	<code>name</code> , <code>score</code>	Representa o jogador humano (dados de identificação e pontuação da partida atual).
<code>Board</code>	<code>size</code> , <code>grid</code> (matriz 2D de <code>Cell</code>), <code>ships</code>	Guarda o estado do tabuleiro; posiciona navios; recebe ataques; verifica se todos os navios afundaram.
<code>Ship</code>	<code>type</code> , <code>size</code> , <code>cells_occupied</code> , <code>hits</code>	Sabe se está afundado (<code>sunk?</code>) comparando <code>hits</code> com <code>cells_occupied</code> .

Classe	Atributos principais	Responsabilidade
<code>Cell</code>	<code>state (:water , :ship , :hit , :miss)</code>	Unidade mínima do tabuleiro.
<code>Weapon</code> (interface Strategy) + <code>MissileWeapon</code> , <code>AirplaneWeapon</code>	<code>target_cells(board, x, y, orientation)</code>	Calcula quais células serão atingidas: míssil = bloco 2×2 ao redor do clique; avião = linha ou coluna inteira. Usar Strategy aqui evita <code>if/case</code> espalhado pelo <code>Board</code> .

Por que “Weapon” como Strategy e não como atributo simples do Ship/Board?

Porque a única variação real do sistema (2 armas com lógica de área diferente) se encaixa perfeitamente no padrão Strategy: o `Board` apenas chama `weapon.target_cells(...)` , sem precisar saber qual arma é. Isso é o único padrão de projeto “extra” do sistema, e se justifica.

4.2 Controllers

Classe	Responsabilidade
<code>GameController</code>	Recebe cliques do mouse na <code>GameView</code> , decide se é posicionamento ou ataque, chama <code>Board / Ship / Weapon</code> , atualiza <code>Game</code> .
<code>MenuController</code>	Navega entre telas (Menu → Setup → Game → Ranking → Instruções).

4.3 Views

Classe	Responsabilidade
<code>MainWindow</code>	Janela Gosu principal; delega <code>update / draw / button_down</code> para a view ativa.
<code>MenuView</code>	Tela inicial (Iniciar, Ranking, Instruções, Sair).
<code>SetupView</code>	Escolha do mapa (Poça/Lago/Oceano) e do modo de posicionamento (manual/automático).
<code>GameView</code>	Desenha os tabuleiros e captura cliques de ataque/posicionamento.
<code>RankingView</code>	Lista o ranking salvo no SQLite, filtrado por tipo de mapa.
<code>InstructionsView</code>	Tela estática com as regras do jogo.

4.4 Utils

Classe	Responsabilidade
<code>Database</code>	Encapsula a conexão SQLite3 e os métodos SQL (<code>insert_match</code> , <code>top_scores(map_type)</code>). Não é um ORM — apenas métodos simples com SQL puro.
<code>ScoreCalculator</code>	Calcula pontuação final a partir de acertos, erros e navios afundados.

4.5 Classes descartadas (e por quê)

- **Ranking (model separado):** desnecessário — o ranking é apenas uma consulta ordenada sobre a tabela `matches` . Criar uma classe/tabela própria duplicaria dado.

- **Repository / DAO por entidade:** a classe `Database` já resolve isso em poucas dúzias de linhas; múltiplos repositórios seriam over-engineering para este escopo.
- **Turn (classe própria):** vira apenas um atributo inteiro/booleano dentro de `Game`.

5. Banco de Dados (SQLite3, sem ORM)

Duas tabelas bastam. Não criamos uma tabela `ranking` separada porque o ranking é derivado de `matches` (`ORDER BY score DESC WHERE map_type = ?`), evitando dado duplicado e sincronização manual.

```
CREATE TABLE players (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT NOT NULL
);

CREATE TABLE matches (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  player_id   INTEGER NOT NULL REFERENCES players(id),
  map_type    TEXT NOT NULL,          -- 'poca' | 'lago' | 'oceano'
  result      TEXT NOT NULL,          -- 'vitoria' | 'derrota'
  score       INTEGER NOT NULL,
  played_at   DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Campo	Explicação
<code>players.id</code>	Chave primária autoincremento.
<code>players.name</code>	Nome digitado ao final da partida.
<code>matches.player_id</code>	Vincula a partida a um jogador.
<code>matches.map_type</code>	Necessário porque cada mapa tem seu próprio ranking .
<code>matches.result</code>	Vitória ou derrota — usado em telas de histórico, se necessário.
<code>matches.score</code>	Pontuação calculada pelo <code>ScoreCalculator</code> .
<code>matches.played_at</code>	Data/hora, útil para desempate ou histórico.

Consulta de ranking por mapa:

```
SELECT players.name, matches.score
FROM matches
JOIN players ON players.id = matches.player_id
WHERE matches.map_type = ?
ORDER BY matches.score DESC
LIMIT 10;
```

6. UML Textual

```
class Game
- board_player: Board
- board_enemy: Board
- current_turn: Symbol
- state: Symbol
+ attack(x, y): Symbol
+ victory?(): Boolean
+ defeat?(): Boolean

class Board
- size: Integer
- grid: Cell[][]
- ships: Ship[]
+ place_ship(ship, coordinates)
+ place_ships_automatically()
+ receive_attack(x, y, weapon): Symbol
+ all_ships_sunk?(): Boolean

class Ship
- type: Symbol
- size: Integer
- cells_occupied: Coordinate[]
- hits: Integer
+ sunk?(): Boolean

class Cell
- state: Symbol

class Weapon (interface)
+ target_cells(board, x, y, orientation): Coordinate[]

class MissileWeapon
implements Weapon

class AirplaneWeapon
implements Weapon

class Player
- name: String
- score: Integer

class GameController
- game: Game
+ handle_click(x, y)
+ next_turn()

class MenuController
+ navigate_to(view)

class Database
+ insert_match(player_id, map_type, result, score)
+ top_scores(map_type): Array

class ScoreCalculator
+ calculate(hits, misses, ships_sunk): Integer

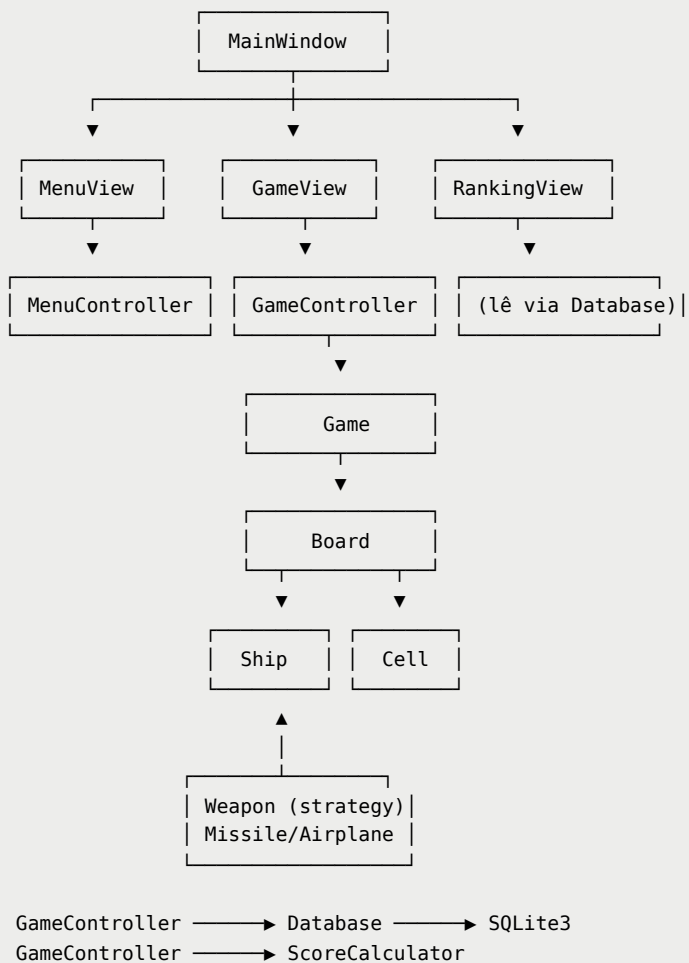
Relacionamentos:
Game "1" *-- "2" Board      (composição: sem Board não há Game)
Board "1" *-- "N" Cell      (composição)
```

```

Board "1" *-- "N" Ship      (composição)
Ship "1" o-- "N" Cell       (associação: ocupa células do Board)
Weapon <|-- MissileWeapon   (herança/interface)
Weapon <|-- AirplaneWeapon  (herança/interface)
GameController --> Game      (usa)
GameController --> Database  (usa, ao final da partida)
ScoreCalculator <.. GameController (dependência)

```

7. Diagrama de Dependências (ASCII)



8. Roadmap — 5 Semanas

Semana	Objetivos	Funcionalidades	Responsável sugerido	Dependências
1	Ambiente e esqueleto	Setup Ruby/Gosu/SQLite3, Gemfile, estrutura de pastas, <code>schema.sql</code> , tela de menu estática	Todos (setup) + Pessoa 1 (menu)	Nenhuma
2	Modelos de domínio	<code>Board</code> , <code>Cell</code> , <code>Ship</code> , escolha de mapa, quantidade automática de navios, posicionamento manual/automático	Pessoa 2 (models) + Pessoa 1 (SetupView)	Semana 1
3	Mecânica principal	Ataques via mouse, validação de coordenadas, turnos, condição de vitória/derrota, armas especiais (Strategy), extra tiro ao acertar	Pessoa 2 (lógica) + Pessoa 1 (GameView)	Semana 2
4	Persistência e pontuação	<code>ScoreCalculator</code> , cadastro de nome, gravação no SQLite, <code>RankingView</code> por mapa	Pessoa 3 (banco) + Pessoa 1 (RankingView)	Semana 3
5	Integração e entrega	Testes manuais ponta a ponta, correção de bugs, README, ensaio da apresentação	Pessoa 4 (integração/testes/doc) + todos	Semanas 1–4

9. Divisão da Equipe

Pessoa	Foco
Pessoa 1	Interface gráfica (Gosu): <code>MainWindow</code> , <code>MenuView</code> , <code>SetupView</code> , <code>GameView</code> , <code>RankingView</code> , <code>InstructionsView</code> .
Pessoa 2	Mecânica do jogo: <code>Game</code> , <code>Board</code> , <code>Ship</code> , <code>Cell</code> , <code>Weapon</code> (Strategy), regras de turno/vitória/derrota.
Pessoa 3	Banco de dados: <code>schema.sql</code> , <code>Database</code> , <code>ScoreCalculator</code> , testes de persistência.

Pessoa	Foco
Pessoa 4	Integração, testes manuais, documentação (README, comentários), preparação da apresentação.

Como evitar conflitos no Git:

- Cada pessoa trabalha majoritariamente em arquivos da sua própria pasta (`views/` , `models/` , `database/`) — reduz conflitos de merge por natureza.
- Commits pequenos e frequentes, com mensagens claras.
- Sincronizar (`git pull origin develop`) antes de iniciar qualquer nova tarefa do dia.
- Combinar previamente quem mexe em `main.rb` e nos `controllers/` (pontos de integração entre partes) para evitar dois integrantes editando o mesmo arquivo ao mesmo tempo.

10. Git Flow Simplificado

Branches:

- `main` — sempre estável, só recebe merge do `develop` já testado (ex: ao fim de cada semana).
- `develop` — branch de integração contínua do time.
- `feature/interface` — Pessoa 1.
- `feature/game` — Pessoa 2.
- `feature/database` — Pessoa 3.

Quando abrir Pull Request:

- Ao terminar uma funcionalidade testada localmente (ex: “posicionamento automático de navios funcionando”).
- PR sempre de `feature/*` → `develop` , nunca direto para `main` .
- Pelo menos 1 outro integrante revisa antes do merge (revisão rápida, não precisa ser formal).
- Merge de `develop` → `main` apenas em marcos (fim de semana de desenvolvimento ou entrega).

11. Checklist de Desenvolvimento

Configuração do ambiente

- [] Ruby instalado e versão combinada entre o grupo
- [] Gemfile com `gosu` e `sqlite3`
- [] Repositório Git criado com branches `main` / `develop`

Interface

- [] Menu inicial (Iniciar, Ranking, Instruções, Sair)
- [] Tela de escolha de mapa e modo de posicionamento
- [] Tabuleiro desenhado a partir da matriz 2D
- [] Interações de mouse (posicionar e atacar)

Lógica

- [] Validação de coordenadas

- [] Quantidade automática de navios por mapa
- [] Posicionamento manual e automático
- [] Armas especiais (Míssil, Avião) via Strategy
- [] Extra tiro ao acertar
- [] Condição de vitória e derrota

Banco

- [] Schema SQLite criado (`players` , `matches`)
- [] Gravação de partida ao final do jogo
- [] Consulta de ranking por tipo de mapa

Testes

- [] Teste manual de cada mapa (Poça, Lago, Oceano)
- [] Teste de posicionamento manual e automático
- [] Teste de vitória e derrota
- [] Teste de gravação/consulta no banco

Documentação

- [] README com instruções de instalação/execução
- [] Comentários no código
- [] Este documento de arquitetura anexado à entrega

Apresentação

- [] Roteiro de demonstração (fluxo: menu → setup → jogo → ranking)
- [] Ensaio com o grupo

12. Pontos a confirmar com o professor/enunciado

Comparando com o PDF oficial ("Etapa 2"), há duas diferenças que vale alinhar antes da entrega final:

1. **Entrada do jogador:** o PDF oficial pede que o jogador informe linha/coluna; seu escopo usa mouse. Mouse é uma evolução válida (ainda resulta em coordenadas linha/coluna internamente), mas confirme se isso é aceito como substituto.
2. **Condição de derrota:** o PDF oficial define derrota por "tentativas esgotadas"; seu escopo não define esse limite explicitamente. Este documento assume um modelo **jogador vs. computador**, onde a derrota ocorre quando todos os navios do jogador são afundados pelo adversário — o que é compatível com "condição de derrota" mencionada nas suas diretrizes, mas é uma decisão de design que vale confirmar (ao invés de derrota por tentativas).

Fora esses dois pontos, seu escopo atende ou supera o mínimo do PDF (tamanho mínimo 5×5 , quantidade de navios , banco relacional , ranking , matriz 2D ).